

Interview Questions Java

Generated on February 05, 2026 at 01:29 PM | 46 entries

Q1. What is the difference between JDK, JRE, and JVM?

[EASY] | Category: Java

Answer: Think of it like a car factory. **JVM (Engine):** This runs the code. It is the engine inside the car. **JRE (Car):** This is the engine plus the body and wheels. It lets you *drive* the car (run the program). **JDK (Factory):** This is the factory. It has the tools to *build* the car (compiler), fix the car (debugger), and drive the car (JRE). To just run Java, you need JRE. To write Java, you need JDK.

Q2. Explain `public static void main` method.

[EASY] | Category: Java

Answer: This is the front door of your program. **public:** Anyone can enter (the computer needs to find it). **static:** It exists even before you build the house (no object needed). **void:** It doesn't give anything back when it finishes. **main:** The specific name on the doorplate. **String args[]:** A mailbox for any messages you send when starting the program.

Q3. What are the 4 pillars of OOP?

[EASY] | Category: Java

Answer: **1. Encapsulation (Protection):** Like a capsule. You hide the medicine inside. You protect your data with `private` and only let people touch it safely. **2. Inheritance (Family):** A child gets traits from parents. A `Dog` class gets code from `Animal` class. **3. Polymorphism (Many Forms):** One button, different actions. Pressing "Play" on a CD player is different from pressing "Play" on a DVD player, but the button looks the same. **4. Abstraction (Hiding Details):** Driving a car. You use the wheel and pedals. You don't need to know how the engine combustion works.

Q4. Difference between `==` and `.equals()`?

[EASY] | Category: Java

Answer: `==` (Address Check): Checks if two things are the *exact same physical object*. Imagine two identical twin brothers. `==` checks "Is this the exact same person?". `.equals()` (Content Check): Checks if two things *look* the same. `.equals()` checks "Do they look exactly alike?". Always use `.equals()` for Strings and Objects.

Q5. Why is String immutable (unchangeable)?

[EASY] | Category: Java

Answer: **1. Safety:** Strings are used for passwords and file paths. If they could change, a hacker could sneak in and change a filename after you checked it. **2. Space:** Java saves space by reusing Strings. If you write "Hello" in 5 places, Java only keeps one copy in memory. If one person changed it, everyone else's "Hello" would break. **3. Speed:** Since it never changes, Java can memorize (cache) its "fingerprint" (hashcode) so looking it up is super fast.

Q6. What are Wrapper Classes?

[EASY] | Category: Java

Answer: Java has two types of data: 1. **Primitives (Simple):** `int`, `char`. Fast but dumb. 2. **Objects (Smart):** `Integer`, `Character`. Slow but powerful. Lists like `ArrayList` only accept Smart Objects. Wrapper classes are simply the "Smart" version of the "Simple" data types so you can put numbers in a List. * `int` -> `Integer` * `char` -> `Character`

Q7. What is the `final` keyword?

[EASY] | Category: Java

Answer: It means "Cannot Change". * **Final Variable:** Like writing in permanent marker. Once you write it, you can't erase or change it. * **Final Method:** "My rules are final." A child class cannot change (override) this method. * **Final Class:** "No kids allowed." You cannot create a child class from this class (like the `String` class).

Q8. What is a Constructor?

[EASY] | Category: Java

Answer: A Constructor is a special setup method. When you buy a new phone, you turn it on and set the language and time. That is the **Constructor**. It runs automatically when you create a new object (`new Phone()`). It sets up the starting values.

Q9. StringBuffer vs StringBuilder?

[EASY] | Category: Java

Answer: Both help you change text without wasting memory. * **StringBuffer:** Think of it like a public notebook where only one person can write at a time. It is safe (thread-safe) but slow because everyone waits in line. * **StringBuilder:** Like a whiteboard where everyone writes at once. It is fast, but if two people write in the same spot, it gets messy. **Rule:** Use `StringBuilder` 99% of the time because it is faster.

Q10. What is the `static` keyword?

[EASY] | Category: Java

Answer: **Static** means "Shared by Everyone". * **Instance Variable:** Every person has their *own* name. * **Static Variable:** Everyone in the room shares the *same* air conditioner temperature. If one person changes it, it changes for everyone. Use `static` for things that should be the same for all objects, like a counter or a constant.

Q11. Overloading vs Overriding?

[EASY] | Category: Java

Answer: **Overloading** (Same Name, Different Options): Like ordering coffee. * `order()` -> Regular coffee. * `order(sugar)` -> Sweet coffee. * `order(milk, sugar)` -> White coffee. Same action name, different ingredients. **Overriding** (New Behavior): Parent says "Go to sleep at 9 PM". Child says "I will go to sleep at 11 PM". The child *replaces* the parent's rule with their own version.

Q12. What is the Collections Framework?

[EASY] | *Category: Java*

Answer: It is a toolbox of containers to store data. **List:** Like a grocery list. Order matters. Duplicates allowed. (`ArrayList`) **Set:** Like a bag of marbles. Order doesn't matter. No duplicates allowed. (`HashSet`) **Map:** Like a dictionary. You look up a word (Key) to find meaning (Value). (`HashMap`)

Q13. What is an Interface?

[EASY] | Category: Java

Answer: An Interface is a **Contract** or a **Menu**. It lists *what* options are available (e.g., "Grill", "Fry"), but it doesn't do the cooking. Any class that signs the contract (implements interface) *must* do the work. Example: `RemoteControl` interface has a button `powerOn()`. The `TV` class provides the code to actually turn on.

Q14. Abstract Class vs Interface?

[EASY] | Category: Java

Answer: **Abstract Class (Partial Blueprint):** Imagine a blueprint for a House. It has walls and a roof (code), but the "Paint Color" is blank. You *must* fill in the blank to build it. Used for related things (Dog, Cat). **Interface (Role):** Imagine a "Swimmer" badge. A Dog can be a Swimmer. A Human can be a Swimmer. A Boat can be a Swimmer. They are not related, but they all do the same thing. Used for abilities.

Q15. What is `this` keyword?

[EASY] | Category: Java

Answer: `this` means "Me" or "My". Imagine a boy named Tom holding a ball. If someone says "Tom, throw the ball", Tom asks "Which ball? The one on the floor or the one in my hand?" `this.ball` specifically means "The ball in *my* hand". It is used to distinguish your own variables from other variables with the same name.

Q16. What is `super` keyword?

[EASY] | Category: Java

Answer: `super` means "My Parent". If you are writing code in a Child class, and you want to use the Parent's code: `* super.money` -> Access parent's variable. `* super.cook()` -> Call parent's method. It is mostly used to call the Parent's constructor to make sure the parent is set up correctly before the child is.

Q17. Exception Handling keywords?

[EASY] | Category: Java

Answer: **Try:** "Attempt to do this risky thing" (like reading a file). **Catch:** "If it fails, do this instead" (like showing an error message). **Finally:** "No matter what happens, do this" (like closing the file). **Throw:** "I found an error! catch this!" **Throws:** "Warning: This method is dangerous."

Q18. Java Access Modifiers?

[EASY] | *Category: Java*

Answer: Think of your house keys. ****Private:**** Only **you** have the key. (Only this class). ****Default:**** Everyone in your **family** has a key. (Same package). ****Protected:**** Family + children living away have keys. (Same package + Subclasses). ****Public:**** The front door is open. **Everyone** can enter. (All classes).

Q19. Checked vs Unchecked Exceptions?

[EASY] | Category: Java

Answer: **Checked (The Nanny):** The compiler checks on you. "Did you pack your lunch?" "Did you handle the File Error?" You *must* handle these (like `IOException`) or the code won't run. Used for external problems. **Unchecked (The Accident):** The compiler trusts you. But you might slip and fall (like `NullPointerException`). These are usually bugs in your code. You aren't forced to catch them, but you should fix the bug.

Q20. Break vs Continue?

[EASY] | Category: Java

Answer: Imagine you are counting cards 1 to 10. **Break:** "Stop counting!" You throw away the deck and walk away. (Exits loop completely). **Continue:** "Skip this card." You ignore card #5 and move straight to card #6. (Skips current turn, keeps looping).

Q21. How does HashMap work?

[MEDIUM] | Category: Java

Answer: It uses a system of **Buckets**. 1. You give it a Key ("Apple"). 2. It calculates a code (Hash) for "Apple". 3. This code tells it exactly which bucket to look in (e.g., Bucket #5). 4. It puts the value in Bucket #5. **Collision:** If "Banana" also lands in Bucket #5, they share the bucket using a list (LinkedList). **Speed:** Since math tells us exactly which bucket to check, we don't need to search the whole room. It's super fast ($O(1)$).

Q22. ArrayList vs LinkedList?

[MEDIUM] | Category: Java

Answer: **ArrayList (Bookshelf):** Books are packed tight. * Finding a book is fast (just count to #5). * Adding a book in the middle is slow (you have to shift *every* other book). **LinkedList (Treasure Hunt):** Each book has a note pointing to the next book. * Finding a book is slow (you must follow the clues from the start). * Adding a book is fast (just change one note to point to the new book).

Q23. Contract between hashCode and equals?

[MEDIUM] | *Category: Java*

Answer: Imagine storing files in folders. **hashCode():** Determines which folder to put the file in. **equals():** Checks if two files inside the folder are identical. **The Rule:** If two files are identical (`equals`` is true), they **MUST** be in the same folder (`hashCode`` is same). If you break this rule, you might put a file in Folder A, but look for it in Folder B and never find it.

Q24. Interface vs Abstract Class (Java 8)?

[MEDIUM] | *Category: Java*

Answer: **Abstract Class:** "I am an Animal." * Can have body parts (variables). * Can have a constructor (birth). * You can only be ONE thing (a Dog cannot be a Cat). **Interface:** "I can Fly." * Cannot have body parts (no variables). * No constructor. * You can do MANY things (a Bird can Fly, Eat, and Sing). * **Java 8 Update:** Interfaces can now have instructions (`default`` methods), not just empty titles.

Q25. Try-with-resources?

[MEDIUM] | Category: Java

Answer: Before Java 7, if you opened a file, you had to remember to close it manually. If you forgot, the file stayed locked (Memory Leak). **Try-with-resources** is an automatic door closer. `try (OpenFile) { ... }` When the code finishes, Java **automatically** closes the file for you. No memory leaks.

Q26. Philosophy of Checked vs Unchecked?

[MEDIUM] | Category: Java

Answer: **Checked:** Recoverable. "The internet is down." -> You can catch this and try again later. **Unchecked:** Fatal. "You tried to access index 10 in a list of size 5." -> This is a logic bug. You can't "recover", you must fix the code. **Modern Trend:** Many programmers hate Checked exceptions because they make code messy, so newer frameworks use Unchecked more.

Q27. Runnable vs Callable?

[MEDIUM] | Category: Java

Answer: They represent a job for a Thread. **Runnable:** "Go run this." * It does the job and comes back empty-handed (`void`). * It cannot complain (throw exception). **Callable:** "Go calculate this." * It does the job and **returns a result**. * If it fails, it brings back an Error (Exception). * Used when you need an answer back.

Q28. What is `volatile`?

[MEDIUM] | Category: Java

Answer: It prevents caching. In a multi-core computer, each CPU has a private notepad (Cache). If Variable A is `volatile`, the CPU is forced to ignore its notepad and write directly to the main whiteboard (RAM). This ensures that if Thread 1 changes the value, Thread 2 sees the new value immediately.

Q29. Stack vs Heap Memory?

[MEDIUM] | Category: Java

Answer: **Stack (Scratchpad):** * Stores temporary variables and method calls. * Very fast. * Thread-safe (my scratchpad is mine). * Cleared automatically when method ends. **Heap (Warehouse):** * Stores actual Objects (`new String()`). * Big storage. * Shared by everyone (Threads can fight over objects). * Cleaned by the Garbage Collector.

Q30. Garbage Collection process?

[MEDIUM] | *Category: Java*

Answer: Java has an automatic cleaner. 1. **Mark:** The cleaner walks through memory and puts a sticker on everything you are still using. 2. **Sweep:** It looks for things *without* a sticker (Garbage) and deletes them. 3. **Compact:** It pushes the remaining objects together to make space for new stuff. You don't have to delete objects manually like in C++.

Q31. Java 8 Streams?

[MEDIUM] | Category: Java

Answer: Think of a factory assembly line. ****Collection:**** A warehouse full of parts. ****Stream:**** The conveyor belt moving the parts. You can attach robots to the belt: * ``filter``: Remove broken parts. * ``map``: Paint the parts. * ``count``: Count the finished parts. The original warehouse is untouched. The Stream just processes the flow.

Q32. What is Optional?

[MEDIUM] | Category: Java

Answer: It is a box that might be empty. Before, if a method returned ``null``, your code would crash (``NullPointerException``). Now, it returns an ``Optional`` box. You are forced to ask: "Is the box empty?" before opening it. It makes your code safer by preventing Null crashes.

Q33. Synchronized Block vs Method?

[MEDIUM] | Category: Java

Answer: ****Method:**** Locking the whole house. If one person is using the bathroom, nobody can enter the `*house*`. Safe, but very slow. ****Block:**** Locking just the door. If one person is using the bathroom, people can still use the kitchen. ``synchronized(bathroom) { ... }``. Much faster because it blocks less code.

Q34. Serialization and serialVersionUID?

[MEDIUM] | Category: Java

Answer: ****Serialization:**** Freezing an object into a file so you can transport it or save it. ****Deserialization:**** Thawing it back into live code. ****serialVersionUID:**** This is the Version Number. If you save a ``User`` object (Version 1), then update your code to add a new field (Version 2), Java checks the ID. If `ID 1 != ID 2`, Java refuses to load the old object because it doesn't match the new code.

Q35. What is ThreadLocal?

[MEDIUM] | Category: Java

Answer: It gives every thread its own private pocket. Normally, variables are shared. ``ThreadLocal`` creates a separate copy of the variable for each thread. ****Use Case:**** User ID. Instead of passing ``userId`` to every single function, you put it in ``ThreadLocal``. Any function in that thread can pull it out of its pocket.

Q36. Comparator vs Comparable?

[MEDIUM] | Category: Java

Answer: **Comparable (Internal):** The object knows how to sort itself. A `Student` class says "I sort myself by ID". This is the default. **Comparator (External):** A separate judge decides the order. You create a `NameSorter` or `AgeSorter`. Use this when you want to sort differently (e.g., sort by Name instead of ID) without changing the original class.

Q37. Why use Generics?

[MEDIUM] | Category: Java

Answer: It adds labels to your containers. **Without Generics:** A list is a mystery bag. You pull something out, you don't know if it's a String or Integer. You might crash if you guess wrong. **With Generics:** `List<String>`. The label says "Strings Only". The compiler stops you from putting an Integer inside. No crashes at runtime.

Q38. What is Reflection?

[MEDIUM] | Category: Java

Answer: It is Java looking in the mirror. Normally, code is fixed. **Reflection** allows code to inspect itself at runtime. It can say: "What methods do I have?" or "Run this method name I found in a string". **Use Case:** Frameworks like Spring use it to build your app automatically without you writing boilerplate code.

Q39. Singleton Pattern?

[MEDIUM] | Category: Java

Answer: Ensures only **ONE** instance of a class exists. Like the President. There is only one. Everyone refers to the same President. **How:** 1. Make the constructor `private` (no one can create new ones). 2. Create one `static` instance inside. 3. Provide a public method `getInstance()` to share it.

Q40. Fail-Fast vs Fail-Safe?

[MEDIUM] | Category: Java

Answer: **Fail-Fast (Strict):** If you try to edit a list while reading it, it crashes immediately (`ConcurrentModificationException`). "Stop! You messed up." Example: `ArrayList`. **Fail-Safe (Chill):** If you edit a list while reading it, it ignores your changes or works on a copy. "Whatever, I'll keep reading the old version." Example: `ConcurrentHashMap`.

Q41. How does ClassLoader work?

[HARD] | Category: Java

Answer: It uses a **Delegation Model** (Ask your parent first). When you need a class: 1. **App Loader:** Ask parent (Extension). 2. **Extension Loader:** Ask parent (Bootstrap). 3. **Bootstrap Loader:** Checks core Java libraries (`java.lang`). If found, returns it. 4. If not found, it tells the child "You look for it". This prevents hackers from replacing core Java classes (like `String`) with their own virus code.

Q42. ConcurrentHashMap internals?

[HARD] | *Category: Java*

Answer: It is a thread-safe Map. **Old Way (Java 7):** Divided map into 16 zones. Only 16 threads could write at once. **New Way (Java 8):** No zones. Uses **CAS** (Compare-And-Swap). * If a bucket is empty, it uses a fast atomic swap (no locks). * If a bucket has data, it locks *only that one bucket*. Result: Thousands of threads can write at the same time without waiting.

Q43. CompletableFuture vs Future?

[HARD] | *Category: Java*

Answer: **Future (Old):** Like a laundry ticket. You have to wait at the counter (`.get()`) until it's done. You block your day waiting. **CompletableFuture (New):** Like a delivery service. You say "When laundry is done, deliver it to my house, then order pizza". You can chain tasks (`.thenApply()`) and do other things while waiting. It is non-blocking.

Q44. What is a Memory Leak?

[HARD] | *Category: Java*

Answer: It happens when you store objects in a list (like a static Map) and forget to remove them. The Garbage Collector sees they are "in use" so it cannot delete them. Over time, the pile grows until the computer runs out of RAM (`OutOfMemoryError`). **Fix:** Use Profiler tools (VisualVM) to find which list is growing forever.

Q45. What is CAS (Compare-And-Swap)?

[HARD] | *Category: Java*

Answer: It is a way to update data without using heavy locks. **Logic:** "I see the value is 5. I want to change it to 6." "Wait, let me check if it is still 5." "If yes -> Update to 6." "If no (someone else changed it) -> Fail and try again." It happens instantly at the CPU level. It is much faster than `synchronized`.

Q46. What is a Dynamic Proxy?

[HARD] | *Category: Java*

Answer: It allows you to create an object that implements an interface on the fly, without writing a class file. **How:** You give it an interface. Java creates a "fake" object. **Why:** When you call a method on the fake object, it routes the call to a handler. **Use Case:** Spring uses this for Transactions. When you call `save()`, the Proxy starts a transaction, calls real `save()`, then commits.